

E14 — Valid Anagram (Hashing & Frequency Count)

Experiment: 14 | LeetCode: #242 | CO: CO2, CO3 | BT Level: BT5

Problem Statement

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

An **anagram** is a word or phrase formed by rearranging the letters of a different word or phrase, using all the original letters exactly once.

Example 1:

Input: `s = "anagram", t = "nagaram"`
Output: `true`

Example 2:

Input: `s = "rat", t = "car"`
Output: `false`

Constraints:

- $1 \leq s.length, t.length \leq 5 * 10^4$
- `s` and `t` consist of lowercase English letters

Follow-up: What if the inputs contain Unicode characters? How would you adapt?

Concept: Anagram Detection via Hashing

Two strings are anagrams if and only if they contain the **same characters with the same frequencies**.

Approaches:

1. **Sorted comparison** — sort both strings and compare: $O(n \log n)$
 2. **Frequency hash map** — count characters in each string and compare: $O(n)$
 3. **Single counter array** — use one fixed-size array (26 for lowercase letters): $O(n)$
-

Solution 1: Frequency Counter (Hash Map)

```
from collections import Counter

def isAnagram_counter(s, t):
    """
    Use Counter (hash map) to compare character frequencies.
```

```
Time: O(n) | Space: O(1) – at most 26 distinct characters
"""
if len(s) != len(t):
    return False
return Counter(s) == Counter(t)
```

Solution 2: Single Frequency Array (Optimal for lowercase ASCII)

```
def isAnagram_array(s, t):
    """
    Use a fixed-size array of 26 integers.
    Increment for s, decrement for t. Check all zeros.
    Time: O(n) | Space: O(1)
    """
    if len(s) != len(t):
        return False

    freq = [0] * 26

    for ch_s, ch_t in zip(s, t):
        freq[ord(ch_s) - ord('a')] += 1
        freq[ord(ch_t) - ord('a')] -= 1

    return all(f == 0 for f in freq)
```

Solution 3: Sorted Strings

```
def isAnagram_sort(s, t):
    """
    Sort both strings and compare.
    Time: O(n log n) | Space: O(n)
    """
    return sorted(s) == sorted(t)
```

Solution 4: Unicode / Follow-Up

For Unicode strings, use a general dict or Counter — same $O(n)$ time, $O(k)$ space where k = number of distinct characters.

```
def isAnagram_unicode(s, t):
    """
    Handles Unicode characters using a defaultdict.
    Time: O(n) | Space: O(k) where k = distinct chars
    """
    from collections import defaultdict

    if len(s) != len(t):
```

```

        return False

    freq = defaultdict(int)

    for c in s:
        freq[c] += 1
    for c in t:
        freq[c] -= 1

    return all(v == 0 for v in freq.values())

```

Detailed Trace

Example 1: $s = \text{"anagram"}, t = \text{"nagaram"}$

Frequency array approach:

Process simultaneously (zip):

```

a/n: freq[a]++ → freq[a]=1,   freq[n]-- → freq[n]=-1
n/a: freq[n]++ → freq[n]=0,   freq[a]-- → freq[a]=0
a/g: freq[a]++ → freq[a]=1,   freq[g]-- → freq[g]=-1
g/a: freq[g]++ → freq[g]=0,   freq[a]-- → freq[a]=0
r/r: freq[r]++ → freq[r]=1,   freq[r]-- → freq[r]=0
a/a: freq[a]++ → freq[a]=1,   freq[a]-- → freq[a]=0
m/m: freq[m]++ → freq[m]=1,   freq[m]-- → freq[m]=0

```

Final freq array: all zeros → return True ✓

Example 2: $s = \text{"rat"}, t = \text{"car"}$

```

Counter(s) = {'r':1, 'a':1, 't':1}
Counter(t) = {'c':1, 'a':1, 'r':1}

```

't' in Counter(s) but not Counter(t): Counter(s) ≠ Counter(t)
 → return False ✓

Hash Table — Collision Concepts

The Counter and dict in Python use a **hash table** internally:

Operation Average Case Worst Case (many collisions)

Insert	$O(1)$	$O(n)$
Lookup	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

Python's hash tables use **open addressing with random probing**, making worst-case collisions extremely rare in practice.

Complexity Summary

Approach	Time	Space
Counter (hash map)	$O(n)$	$O(1)$ — 26 chars
Frequency array	$O(n)$	$O(1)$ — fixed 26 slots
Sorted strings	$O(n \log n)$	$O(n)$
Unicode hash map	$O(n)$	$O(k)$ — k distinct chars

Extension: Group Anagrams (LC #49)

The same frequency-counting idea underlies grouping anagrams: use the sorted string (or a tuple of char counts) as the hash key.

```
from collections import defaultdict

def groupAnagrams(strs):
    groups = defaultdict(list)
    for word in strs:
        key = tuple(sorted(word))    # or tuple of 26-element freq array
        groups[key].append(word)
    return list(groups.values())
```

Key Takeaways

- **Length check first** is a cheap $O(1)$ early exit.
- The **frequency array** approach is fastest in practice for lowercase ASCII — no hash function overhead.
- For Unicode, fall back to `Counter` or `defaultdict`.
- Anagram detection is a classic hash table application demonstrating $O(n)$ vs $O(n \log n)$ trade-offs.